

INTO - CPS (Introduction)

I Cyber-Physical systems (CPS) sono dei sistemi complessi e interconnessi che comprendono interazioni tra elementi elettronici, fisici e umano. Un esempio di CPS è un Line-Follower Robot, che sfrutta sensori di luminosità per trovare le aree più chiare e più scure (la linea) nel terreno. I bisogni di questa disciplina comprendono la multidisciplinarietà, un costo di sviluppo basso così come il time-to-market, l'esplorazione di nuovi design in modo efficiente, garantire una buona tolleranza ai guasti, documentazione per descrivere il lavoro svolto, e sicurezza per gli stakeholder. La Co-simulation è una tecnica che permette l'accoppiamento di diversi simulatori, ognuno dei quali può utilizzare tool e formalismi differenti. La tool-chain INTO-CPS fornisce un ambiente collaborativo per la CPS engineering, offrendo lavoro concorrente e integrazione graduale dei componenti.

Data Alteration Attacks

• **Attacco ai sensori**: vengono corrotti i dati ricevuti dai sensori; l'output finale sarà dunque basato su dati scorretti. Un attacco del genere potrebbe essere confuso con un funzionamento scorretto dell'unità colpita; potrebbe inoltre modificare variabili di controllo private, o anche impattare sui meccanismi di logging e tracing.

• **Attacco agli attuatori**: vengono corrotti i dati inviati agli attuatori, facendo sì che l'output effettivamente calcolato venga ignorato. Il risultato è una significativa alterazione del comportamento del sistema.

Un attacco può essere implementato sia all'interno dell'unità FMU sia all'esterno, agendo sulla comunicazione tra FMU e dispositivi.

Theorem-Prover: Prototype Verification System

Un formal system è un sistema usato per provare la veridicità di frasi tramite deduzioni. Consiste di assiomi (concetti o frasi considerate vere a priori), un insieme di regole di inferenza.

Theorem prover

È un programma che implementa un Formal System, prendendo come input una definizione formale, espressa secondo la logica del sistema, che deve essere verificata, e le proprietà che devono essere dimostrate. Il programma prova dunque a costruire una dimostrazione applicando le regole di inferenza. In particolare, PVS è un Theorem Prover interattivo, implementa un Higher-Order logic language, e il sequent calculus come sistema deduttivo. PVS viene usato in diversi campi, tra cui la verifica formale di Hardware, Software, CPS critici e in real time.

PVS theories

Una specifica PVS è composta da una o più teorie, e permette la definizione di variabili.

Il **Sequent Calculus** lavora su elementi detti **sequent**, e consistono in antecedenti e conseguenti, separati dal simbolo " $\vdash$ ".

Simbologia formale: $A_1, \dots, A_n \vdash B_1, \dots, B_n$

Simbologia informale: $A_1 A_2 \dots A_n \Rightarrow B_1 B_2 \dots B_n$

Il sequent calculus cerca di trovare la dimostrazione partendo dal **goal Sequent** (ciò che si vuole dimostrare) e procede all'indietro, creando un albero dove la root è il goal S. La prova è completa quando (e se) tutti i rami terminano con TRUE.

Un sequent è provato True se almeno una tra queste è verificata:

- almeno un antecedente è False
- almeno un conseguente è True
- compare sia come antecedente che come conseguente una formula nota

PVS utilizza anche:

- Connettivi logici;
- Operatori complessi come if-else;
- Theories: collezioni di definizioni e formule

Commands PVS

- **flatten**: applica semplificazione logica al sequent con ID specificato
- **split**: splits l'elemento ID creando due branch

Quando applicare questi comandi? Dipende da location e connettivo logico

	OR, $\Rightarrow$	AND, IFF	Equivalenze logiche: $P \Rightarrow Q \equiv \overline{P} \vee Q$ $P \text{ iff } Q \equiv (P \Rightarrow Q) \text{ AND } (Q \Rightarrow P)$
Antecedente	split	flatten	
Consequente	flatten	split	

- **skeep**: skolemizzazione
- **inst?**: istanziazione

	OR, $\Rightarrow$	AND, IFF
Antecedente	inst	skeep
Consequente	skeep	inst

La simbologia di altri comandi e le teorie su come risolvere gli esercizi è solo sulle slide
Hardware Analysis sulle slide

Mobius

Software per modellare il comportamento di sistemi complessi (reazioni biochimiche, sicurezza nei computer)

Le principali Feature sono il supporto di molteplici linguaggi di modellazione, modelli gerarchici, misurazioni personalizzabili delle proprietà dei sistemi in analisi, soluzioni numeriche e simulazioni.

Ogni progetto Mobius è composto da diversi moduli:

Atomic Models

Sono composti da sottomodelli, che possono anche essere collegati tra di loro tramite Eventi e Gate.

È possibile anche definire variabili, spesso usate come rappresentazione del rate degli eventi

Composed Models

Permette la composizione tra più moduli precedentemente definiti, tramite gli operatori join e model.

Reward Models

Definiscono il sistema usando Performance Variables, calcolate usando Reward Functions

Study Model

Permette di definire set di valori da assegnare alle variabili globali

Solver

Mobius prevede simulazione o risoluzione numerica per ottenere la misura di interesse. Nel primo caso avremo soluzioni statisticamente accurate, mentre nel secondo soluzioni esatte

Transformer

Converte modelli ad alto livello in rappresentazioni a basso livello basate su stati

Sulle slides (Mobius 2-3-4) sono presenti esercizi